Sampling and Per-Precision Histograms with nixnan

An Experiment on `rd_nixnan.cu`

ganesh@cs.utah.edu

September 24, 2026

The Example, at a Glance

`rd_nixnan.cu`: a 1D reaction-diffusion PDE $\partial u/\partial t = D\,\partial^2 u/\partial x^2 + \lambda u$, explicit FTCS, 101-point grid, 1600 steps — the *same* kernel implemented once each in FP16, BF16, FP32, FP64.

How precision-sensitive is this example?

- The reaction term grows every value by $\sim$1.05$\times$ per step, *regardless of format* — so the only thing that differs between formats is how much exponent range each one has before it overflows.
- FP16's 5-bit exponent runs out at step 300; BF16/FP32/FP64's wider exponents are all still finite at step 1600 — same algorithm, same input, one format fails and three don't.

NixNan Feature Reference: BIN_SPEC_FILE

nixnan's own banner gives this example:

```
{ "count": 128, "bf16": [], "f16": [[0,5],[-4,-1]],
  "f32": [], "f64": [] }
```

- Each format key: a list of *unbiased signed exponent* ranges to watch. Here only `f16` watches anything: $[0, 5]$ and $[-4, -1]$.
- `count`: report every `count`-th occurrence in a watched range — `count:128` = "every 128th hit."
- **Zero and infinity are structurally excluded from every range** — a value that jumps straight from finite to `inf` in one step is invisible to this mechanism, no matter how ranges are drawn. (Matters later: this is exactly what happens here.)

Concept 1: Sampling — The Mechanism

Every run here uses `nixnan.so` directly ("straight nixnan"), no wrapper:

```
LD_PRELOAD=<path>/nixnan.so [ENV_VARS...] ./rd_nixnan
```

`SAMPLING=N` instruments only every *N*th launch of a repeated kernel — a knob trading overhead for observation completeness.

Verified before trusting it: an older comment in this repo claims *"SAMPLING is a no-op."* We re-tested directly (`SAMPLING=1` vs. `SAMPLING=100000`): exception counts dropped to exactly zero at the high value. **Confirmed fixed** in the current build.

Concept 1: Every Call vs. Once in 8

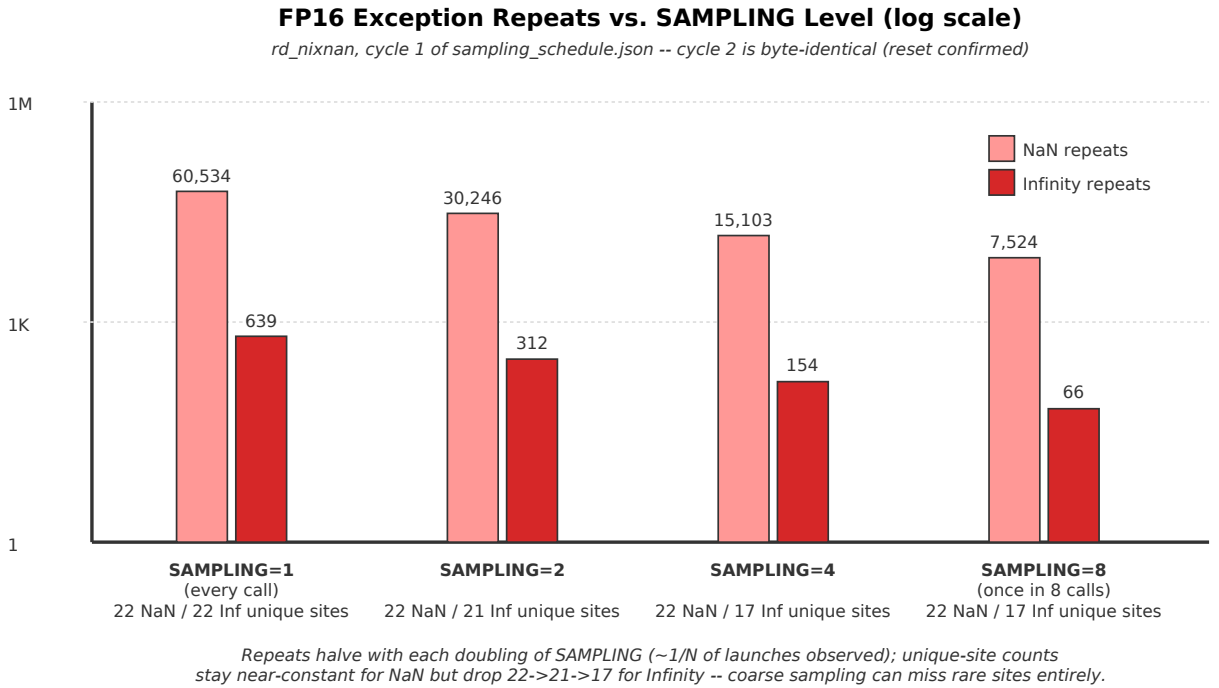
FP16 exception counts (the only format that overflows here):

	NaN (unique / repeats)	Infinity (unique / repe
`SAMPLING=1` (every call)	22 / 60,534	22 /
`SAMPLING=8`	22 / 7,524	17 /

Repeats drop $\sim 8\times$, as expected. But **5 distinct Infinity sites vanish entirely** ($22 \rightarrow 17$) — coarse sampling doesn't just under-count, it can miss whole bugs.

Concept 1: An Exponentially Decaying Schedule

`SAMPLING` is fixed for a whole process run, so a decaying rate is built as a sequence of short runs (`sampling_schedule.json`): step 4 times at spacing 1 $\rightarrow$ double to 2 $\rightarrow$ 4 $\rightarrow$ 8 $\rightarrow$ **reset**.



Every snap at a level is byte-identical; **cycle 2 exactly reproduces cycle 1** — the reset is provably clean. The TSV = 32 independent runs (4 levels $\times$ 4 snaps $\times$ 2 cycles), each just `SAMPLING=$level LD_PRELOAD=nixnan.so ./rd_nixnan` — no other setting changed from the Section 4 baseline.

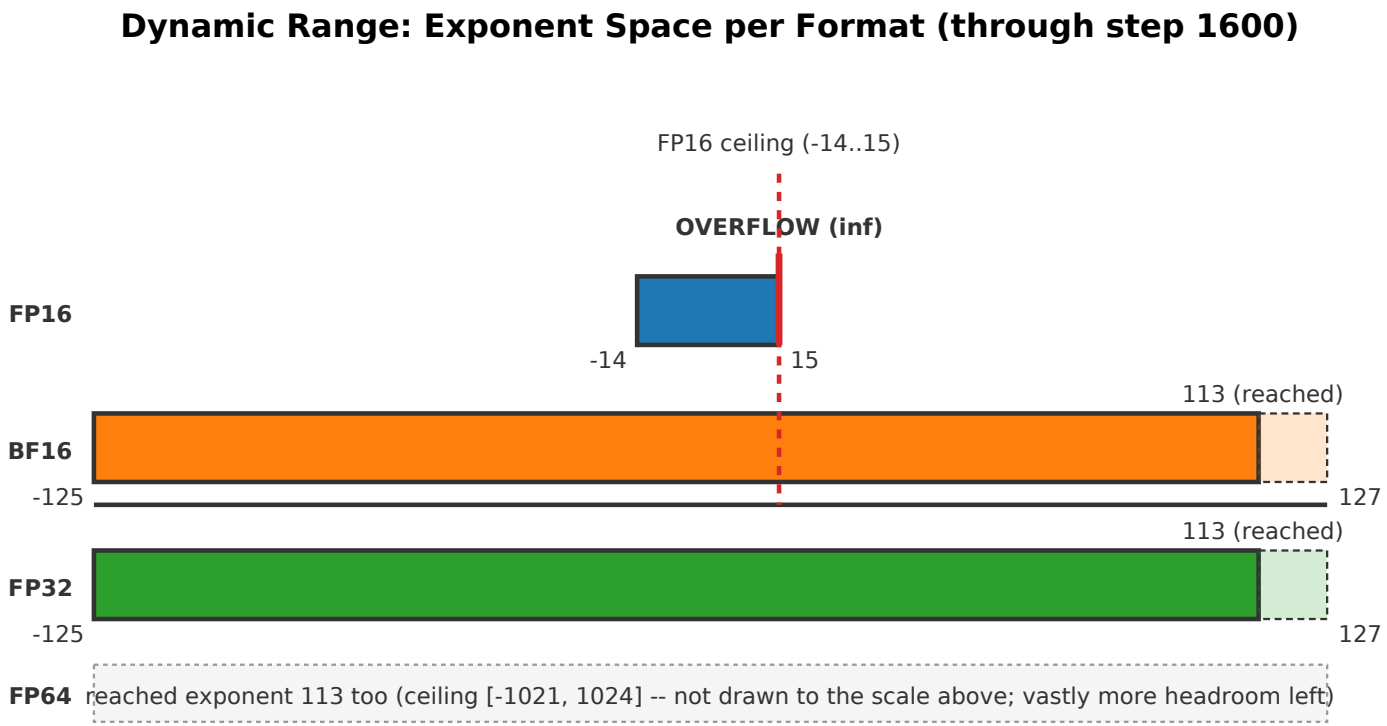
Concept 2: Histograms per Precision — The Mechanism

`HISTOGRAM=1` tracks the exponent of every instrumented FP value, independent of exception checking:

```
HISTOGRAM=1 LD_PRELOAD=nixnan.so ./rd_nixnan
```

At exit, nixnan reports the [min, max] exponent *actually observed* for each format — one number per precision bucket, no per-value log needed.

Concept 2: Results — Which Format Actually Failed



`f16` alone reports `inf`. `bf16`/`f32`/`f64` all reach the *identical* exponent, 113 — same trajectory, different ceilings, only one ceiling reached.

Concept 3: Making BF16 and FP32 Fail Too

Every result so far: FP16 fails, the other three don't. Can we make BF16/FP32 fail as well?

Tried: domain enlargement (bigger N). Does **not** work — the reaction term `lam * u[i]` grows every point by the same factor regardless of N/dx. Already confirmed earlier this session: `rd_nixnan_large.cu` ($10\times$ grid, N=1001) still only failed in FP16 at M=1600.

Works: more time steps. M is a compile-time macro:

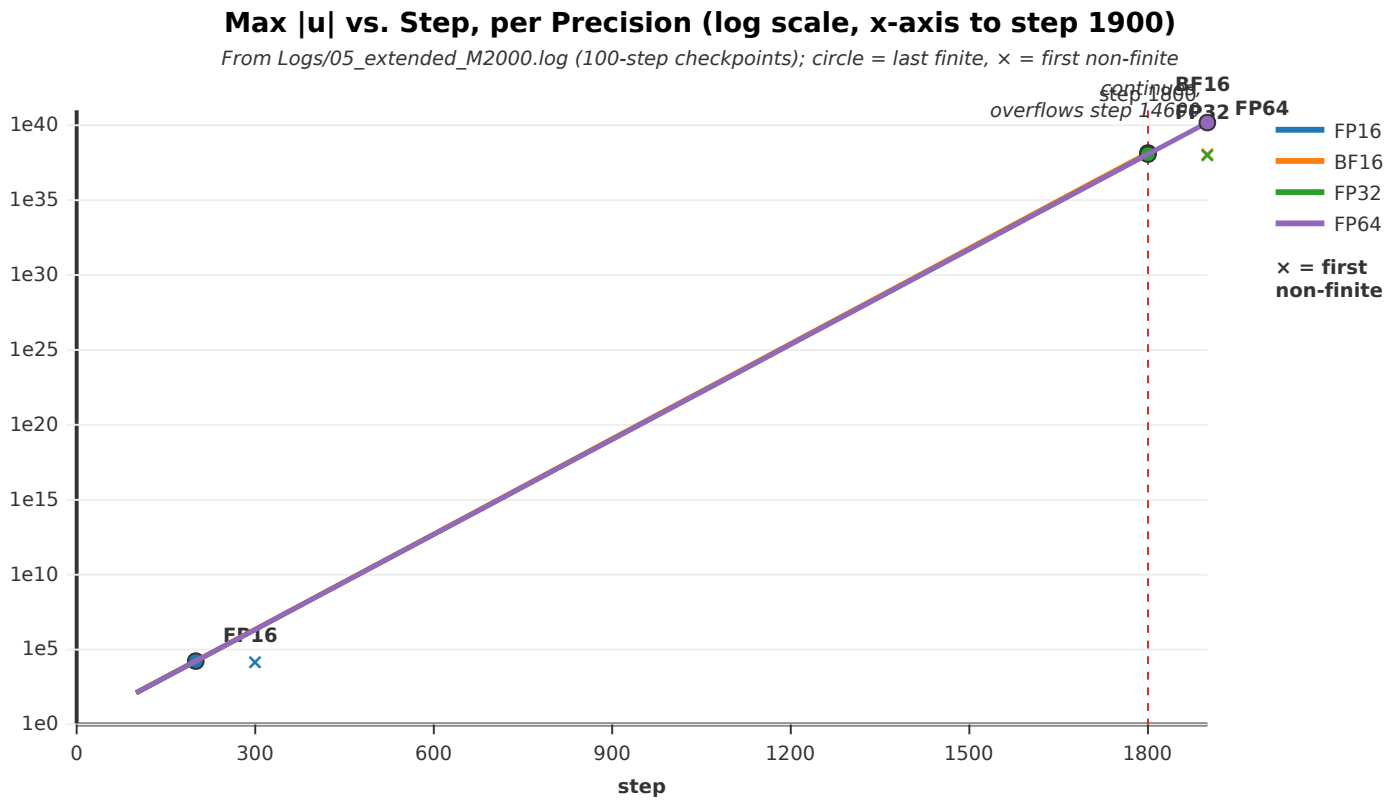
```
nvcc -arch=sm_86 -DM=2000 rd_nixnan.cu -o rd_nixnan_extended
LD_PRELOAD=nixnan.so ./rd_nixnan_extended
```

Concept 3: Results at M=2000

Format	First non-finite	NaN u/r	Inf u/r
FP16	step 300	22 / 78134	22 / 639
BF16	step 1900	15 / 8975	6 / 76
FP32	step 1900	19 / 10966	8 / 90
FP64	— (finite)	0 / 0	

BF16 and FP32 now both show real nixnan-classified exceptions. FP64 correctly stays finite at M=2000 — it does eventually overflow too, much later (next slide).

Concept 3: Where Infinity Actually Comes From



FP16 (blue) departs the shared curve at step 300. BF16/FP32/FP64 stay numerically **indistinguishable** through step 1800 (1.1675×10^{38}, all three, to 4 sig. figs.); BF16/FP32 are non-finite by step 1900, FP64 continues (real value there $\approx 1.5 \times 10^{40}$, still finite).

Concept 3: The Exact CUDA Statement

Rebuilt with `-g -lineinfo` so nixnan's `LINE_INFO=1` could resolve real source locations (not just SASS):

```
error [infinity] detected ... instruction FADD R0, R0, R7
  ; in function rd_step_fp32 at rd_nixnan.cu:96 of type f32
```

Line 96: `float lap = (u[i+1] + u[i-1]) - 2.f * u[i];` — **Infinity first appears in the diffusion term, not the reaction term.** (FP16's `__h*` intrinsics never resolve a line in this build — a real nixnan limitation; same conclusion by structural analogy.)

Inf → NaN, confirmed to the instruction:

```
FADD R0,R0,R7 ; u[i+1]+u[i-1] -> operand already Infinity
FADD R7,R6,R6 ; 2.f*u[i] -> operand already Infinity
FADD R7,R0,-R7 ; (sum) - 2.f*u[i] -> RESULT is NaN (inf - inf)
```

Matches the source file's own long-standing comment (`inf - 2*inf + inf -> NaN`) — now tool-verified, not hand-derived.

FP64: The Real Overflow

FP64's ceiling is $\sim 1.8 \times 10^{308}$; from step 1800's value and the $\sim 1.05\times$/step growth, overflow is expected around step 14,550. A new build mode, `FP64_LONGRUN`, runs all four formats together to step 3800 ($2\times$ FP32's trigger step), then steps FP64 alone until it overflows:

```
nvcc -DFP64_LONGRUN=1 -g -lineinfo rd_nixnan.cu -o rd_nixnan_fp64_longrun
LD_PRELOAD=nixnan.so ./rd_nixnan_fp64_longrun
```

FP64 overflows at **step 14600** — matching the estimate. nixnan now classifies it for real:

	NaN u/r	Inf u/r	−Inf u/r
FP64	36 / 1699	10 / 74	32 / 712

Same statement as FP32, `rd_nixnan.cu:106` (the Laplacian), same `DADD, DADD, DADD`→`NaN` mechanism — just 12,700 steps later, because FP64 has far more exponent range to climb through.

New Feature: Per-Instruction Histogram Attribution

A histogram threshold report used to name only a **format, kernel, and range** — unlike an exception report, which also names the exact instruction. Branch `print_histo_instrn` (nixnan commit `15ad4f8`) closes that gap.

How invoked: a new `BIN_SPEC_FILE` key, `"<fmt> (record_inst)"`, alongside the plain `"<fmt>"` key:

```
{ "count": 1, "f16 (record_inst)": [[-14,-13],[14,15]] }
```

Same `[lower,upper]` range syntax; every bin it defines now also prints the SASS instruction that triggered each hit. Plain `"<fmt>"` keys are unchanged — verified byte-identical before/after.

What it does, internally: every instrumented instruction is registered in the same instruction table the exception path already builds; its id rides along through the histogram channel and is resolved only when the triggering bin's `record_inst` flag is set.

Dribble of Instructions Before the Exception

Unsampled: 11,000+ lines (one per occurrence). Add `SAMPLING=128` (1 in 128 launches instrumented) and the same watched ranges collapse to seven lines, still a legible story:

```
range=[-14,-13] instr=HFMA2 R0, R9.H0_H0, -2, -2, R0.H0_H0
range=[-14,-13] instr=HFMA2 R0, R9.H0_H0, -2, -2, R0.H0_H0
range=[-14,-13] instr=HFMA2 R0, R9.H0_H0, -2, -2, R0.H0_H0
range=[-14,-13] instr=HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0
range=[-14,-13] instr=HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0
range=[-14,-13] instr=HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0
range=[-14,-13] instr=HFMA2 R0, R0.H0_H0, c[0x0][0x170].H0_H0, R9.H0_H0
error [NaN] detected ... instruction HADD2 R0, R0.H0_H0, R7.H0_H0
```

Two `HFMA2` variants feed tiny near-boundary values early on, then silence (no `[14,15]` hits at all), then `NaN` at the same `HADD2` already identified as the Laplacian's overflow point.

Honest tradeoff: the real overflow onset (∼step 227) isn't a multiple of 128, so it's never itself sampled — step 256 (the next sampled launch) already sees `NaN`, not the original `Infinity`. Same `SAMPLING` completeness gap as Concept 1, in a new form: a real, correctly-attributed, but *downstream* exception.

Getting Both Exceptions: Stride, Not Budget

Question: would `MAX_ERRORS=2` recover the Infinity too? Tested directly, both ways:

- Unsampled, `MAX_ERRORS=2`: sites 1 *and* 2 are both `Infinity` (two different instructions). No NaN until ∼site 10.
- `SAMPLING=128`, `MAX_ERRORS=2`: sites 1 *and* 2 are both `NaN`. `Infinity` never appears, at *any* `MAX_ERRORS` — no instrumented launch ever witnesses it. **Stride problem, not a budget problem.**

Fix: tighten SAMPLING, not MAX_ERRORS.

`..._SAMPLING8_MAXERRORS4.sh` (`SAMPLING=8`, `MAX_ERRORS=4`) catches both:

```
error [NaN,infinity] operand 1 of HADD2 R0, R0.H0_H0, R7.H0_H0
error [NaN,infinity] operand 2 of HADD2 R0, R0.H0_H0, R7.H0_H0
error [NaN,infinity] operand 0 of HADD2 R0, R0.H0_H0, R7.H0_H0
error [NaN,infinity] operand 1 of HFMA2 R0, R9.H0_H0, -2, -2, R0.H0_H0
```

Still not a clean "Inf line, then a NaN line": every site is tagged `NaN,infinity` *together* — a warp's 32 lanes are OR'd before reporting, and different grid points are at different points of the same buildup within one launch. Cost: 1,314 histogram lines (vs. 7 at `SAMPLING=128`).

All the Invocation Flags for a Sampled Run

Flag	Purpose
`LD_PRELOAD=nixnan.so`	Load the tool
`SAMPLING=128`	Instrument 1 in 128 launches
`HISTOGRAM=1`	Enable exponent histogramming
`BIN_SPEC_FILE=..._Sampled.json`	`"f16 (record_inst)"` ranges, coun
`MAX_ERRORS=1`	Stop at the first exception

```
SAMPLING=128 HISTOGRAM=1 \
  BIN_SPEC_FILE=Stop_First_Error_with_History_FP16_Sampled.json \
  MAX_ERRORS=1 LD_PRELOAD=nixnan.so ./rd_nixnan
```

Every flag here already existed in nixnan except `BIN_SPEC_FILE`'s new `(record_inst)` key — the whole sampled, per-instruction, stop-at-first-error trace is five flags, no wrapper script or change to `rd_nixnan.cu` needed.

Native Decaying Report Rate: FP32 Fails, FP64 Doesn't

Unlike `SAMPLING` (Concept 1: no native schedule, built one outside nixnan), `BIN_SPEC_FILE` has its own: `doublings` — report every `count` occurrences, double the threshold each report, reset after `doublings` doublings. Ported in (from branch `fp-reset`) and fixed — see `atomicCas_Stdout_Tearing_fixes.tex`.

M=2000 already makes FP32 fail while FP64 stays finite. Watching only FP32's highest two exponents (`[126,127]`) with `count:1`, `doublings:10`:

```
{ "count": 1, "doublings": 10, "f32": [[126, 127]] }
```

`Exp-Output-Rate-FP32-Top2Exp-M2000-start-1-double-10-times.sh` confirms both halves: FP32 hits "first non-finite at step 1900"; FP64's block ends with no such line at all. The reported sequence climbs cleanly through all 11 levels, 1 to 1024 (ten doublings — double the FP16 predecessor's reach of 256), resetting repeatedly for the rest of the run.

What nixnan Made Visible Here (1/4)

1. **Sampling trades overhead for completeness**, not just verbosity — coarse sampling can drop entire exception sites, not only reduce their reported repeat count.
2. **A schedule can be built around a static env var** via orchestration, and — because the program is deterministic — that schedule's reset is independently verifiable.
3. **Per-format exponent histograms turn "FP16 probably fails" into a specific, quantified claim**: FP16 alone overflows; the other three formats are proven to share one common growth trajectory up to a shared checkpoint exponent.

What nixnan Made Visible Here (2/4)

4. **nixnan can adjudicate a hypothesis about why a format fails first**: domain enlargement is ruled out by the kernel's own growth term; running longer is confirmed, with nixnan itself newly reporting real BF16/FP32 exceptions at M=2000 where it reported none at M=1600.
5. **Line-level attribution turns a hand-derived guess into a tool-verified fact**: with `-lineinfo`, nixnan pinpoints the exact statement (the Laplacian) and instruction where $\infty - \infty$ produces `NaN` — confirming a mechanism the source comments could only predict by hand.

What nixnan Made Visible Here (3/4)

6. **Running FP64 long enough shows it fails too, and nixnan shows exactly where**: run for real to step 14600, FP64 overflows at the step its own growth rate predicts, attributed to the identical Laplacian statement and mechanism already confirmed for FP32.
7. **Per-instruction histogram attribution turns "a value landed in this range" into "this specific instruction put it there"**: `"<fmt> (record_inst)"` extends histogram threshold reports with the same instruction-level detail exception reports already carry — and, combined with `SAMPLING`, shows the same completeness gap yet again: a real, correctly- attributed `NaN`, one launch downstream of the original Infinity.

What nixnan Made Visible Here (4/4)

8. **Recovering both requires the right knob, not more of the wrong one**: raising `MAX_ERRORS` alone cannot recover a missed Infinity, at any value — it counts distinct sites, not occurrences. Tightening `SAMPLING`'s stride does, though even then as one combined `NaN,infinity` report (a warp's lanes are classified and OR'd together).
9. **A decaying report rate can be native to nixnan, not just orchestrated around it**: `BIN_SPEC_FILE`'s `doublings` key — ported in and fixed as part of this work — watches FP32's highest two exponents through one M=2000 run where FP32 overflows and FP64 correctly does not, climbing cleanly through 11 threshold levels (1 to 1024) and resetting, no orchestration script required.